

Introduction to Objects and Classes

Java is an **object-oriented programming (OOP)** language. Instead of organizing a program only around functions or procedures, Java allows us to model real-world entities as **objects**.

For example, in a university system, we may have:

- Student
- Teacher
- Course
- Classroom
- Enrollment
- Grade

Each of these can be represented using a Java **class**.

For example:

```
class Student {  
    String name;  
    int age;  
    String studentId;  
}
```

We can then create actual students from this class:

```
Student student1 = new Student();  
Student student2 = new Student();
```

Here:

- Student → class
- student1 → object/**reference**
- student2 → object/**reference**
- new → creates an object

Classes in Java

What is a Class?

A **class** is a blueprint or template used to create objects.

A class defines:

- Data
- Behavior
- Rules for how an object operates

DCIT 50: Object Oriented Programming

Basic syntax:

```
class ClassName {  
  
    // variables  
  
    // methods  
  
}
```

Example:

```
class Student {  
  
    String name;  
    int age;  
  
    void displayInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

The class itself does not represent a particular student. Instead, it describes what every Student object will have.

Objects in Java

What is an Object?

An **object** is an instance of a class. A class defines what an object **has** and **can do**, while an object represents an actual instance of that class.

Real-World Analogy

Consider a blueprint for a house. The **blueprint** describes:

- number of rooms
- doors
- windows
- dimensions

DCIT 50: Object Oriented Programming

But the blueprint itself is not an actual house. From the blueprint, we can construct multiple houses.

Similarly:

Class = Blueprint

Object = Actual instance created from the blueprint

For Example:

```
class Student {  
    String name;  
    int age;  
}
```

The class describes what a Student object contains.

We can create objects:

```
Student student1 = new Student();  
Student student2 = new Student();
```

Now we have two separate objects.

We can assign different values:

```
student1.name = "Juan";  
student1.age = 20;  
  
student2.name = "Maria";  
student2.age = 21;
```

The objects contain different data even though they were created from the same class.

The new Keyword

The new keyword is commonly used to **create an object**.

Example:

```
Student student1 = new Student();
```

Let's break this down:

```
Student student1 = new Student();
```

Student

```
Student
```

DCIT 50: Object Oriented Programming

This is the **data type**, which is the class type.

student1

```
student1
```

This is a **reference variable** that refers to the object.

new

```
new
```

Requests creation of a new object.

Student()

```
Student()
```

Calls the class's constructor.

So:

```
Student student1 = new Student();
```

can conceptually be understood as:

"Create a new Student object and let student1 refer to it."

Objects Have State and Behavior

An object generally has two important characteristics:

State

The **state** represents the data stored by an object.

For a student:

Name: Juan

Age: 20

Student ID: 2026-001

Behavior

The **behavior** represents what the object can do.

For example:

Enroll in a course

View grades

Change password

Display information

DCIT 50: Object Oriented Programming

In Java:

- **Variables/fields** represent state.
- **Methods** represent behavior.

Example:

```
class Student {  
  
    String name;  
    int age;  
  
    void displayInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

Characteristics: Variables and Methods

Variables

Variables inside a class are commonly called **fields** or **instance variables** when they belong to an object.

Example:

```
class Student {  
  
    String name;  
    int age;  
    double grade;  
}
```

The class has three variables:

```
String name;  
int age;  
double grade;
```

Each object gets its own instance values.

DCIT 50: Object Oriented Programming

Example:

```
Student student1 = new Student();
Student student2 = new Student();

student1.name = "Juan";
student2.name = "Maria";
```

The objects have independent values:

```
student1
  name = Juan
```

```
student2
  name = Maria
```

Methods

A **method** defines behavior or an operation that an object can perform.

Example:

```
class Student {

    String name;
    int age;

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

The method is:

```
void displayInfo()
```

We can call it using an object:

```
student1.displayInfo();
```

Method with Parameters

Methods can receive information.

```
void setName(String newName) {
    name = newName;
}
```

DCIT 50: Object Oriented Programming

Usage:

```
student1.setName("Juan");
```

Method with Return Value

A method can also return a value.

```
int getAge() {  
    return age;  
}
```

Usage:

```
int studentAge = student1.getAge();
```

Object

There is an important distinction between a **class** and an **object**.

Class	Object
Blueprint/template	Actual instance
Defines properties	Contains actual values
Defines methods	Can execute methods
Does not represent a specific entity	Represents a specific entity
Example: Student	Example: student1

Important Terminology

Students should remember these terms:

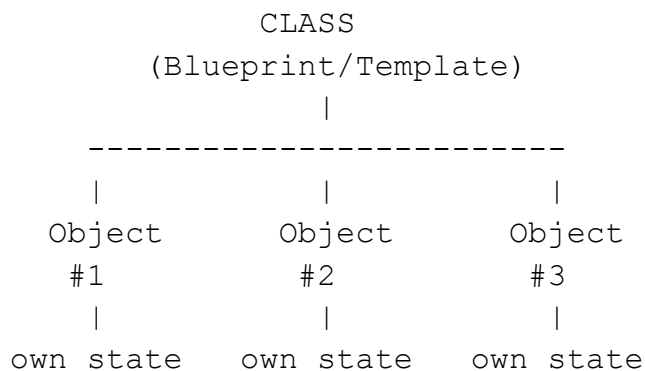
Term	Meaning
Class	Blueprint/template for objects
Object	Instance of a class
Instance	Another term for an object created from a class
Field	Variable belonging to a class/object
Method	Defines behavior/operation
new	Creates a new object
Reference variable	Variable that refers to an object
Instance variable	Variable whose value belongs to a particular object

Quick Review Questions

1. What is a class?
2. What is an object?
3. What keyword is commonly used to create an object?
4. What represents the state of an object?
5. What represents the behavior of an object?
6. Can one class create multiple objects?
7. Do different objects have to contain the same values?

Summary

The fundamental relationship to remember is:



In Java:

```
class Student {
    String name;

    void display() {
        System.out.println(name);
    }
}
```

Create objects:

```
Student s1 = new Student();
Student s2 = new Student();
Initialize them:
s1.name = "Juan";
s2.name = "Maria";
Use their behavior:
s1.display();
s2.display();
```

DCIT 50: Object Oriented Programming

The core concept for this topic:

- ***A class defines what an object is and what it can do.***
- ***An object is an actual instance created from that class.***